

Go-Back-N (GBN) Protocol

A Sliding Window Approach to Reliable Data Transfer

Introduction

The **Go-Back-N (GBN)** protocol is a **pipelined reliable data transfer** mechanism that extends the principles of **Stop-and-Wait (rdt3.0)** to improve efficiency. Instead of sending one packet at a time, GBN allows multiple packets to be in transit before acknowledgments are received.

Key Idea: The sender can have up to N unacknowledged packets in the pipeline. If one packet is lost, the sender “*goes back*” and retransmits that packet and all subsequent ones.

Window Concept and Sequence Numbers

Sliding Window View

The sender maintains two key variables:

- **base:** Sequence number of the oldest unacknowledged packet.
- **nextseqnum:** Sequence number of the next packet to send.

At any given moment:

$[0, \text{base} - 1] \Rightarrow$ Acknowledged packets
 $[\text{base}, \text{nextseqnum} - 1] \Rightarrow$ Sent but not yet ACKed
 $[\text{nextseqnum}, \text{base} + N - 1] \Rightarrow$ Packets allowed to send
 $[\text{base} + N, \infty) \Rightarrow$ Window full

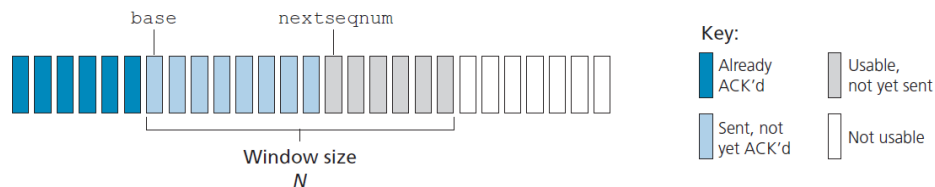


Figure 1: Sender's View of Sequence Numbers in Go-Back-N Protocol

Terminology: The window of size N “slides forward” with each ACK received, hence the name **Sliding Window Protocol**.

Protocol Operation

Sender Behavior

The sender responds to three main events:

1. **Upper Layer Invocation (rdt_send):** If unacknowledged packets $< N$, the sender creates and sends a packet:

$\text{sndpkt}[\text{nextseqnum}] = \text{make_pkt}(\text{nextseqnum}, \text{data}, \text{checksum})$

If the window is full, new data is temporarily refused (or buffered).

2. **ACK Received:** A cumulative ACK for packet n confirms all packets $\leq n$ are received:

$\text{base} = \text{ACKnum} + 1$

3. **Timeout:** On timeout, the sender retransmits all unacknowledged packets:

for $i = \text{base}$ to $\text{nextseqnum}-1$, retransmit $\text{sndpkt}[i]$

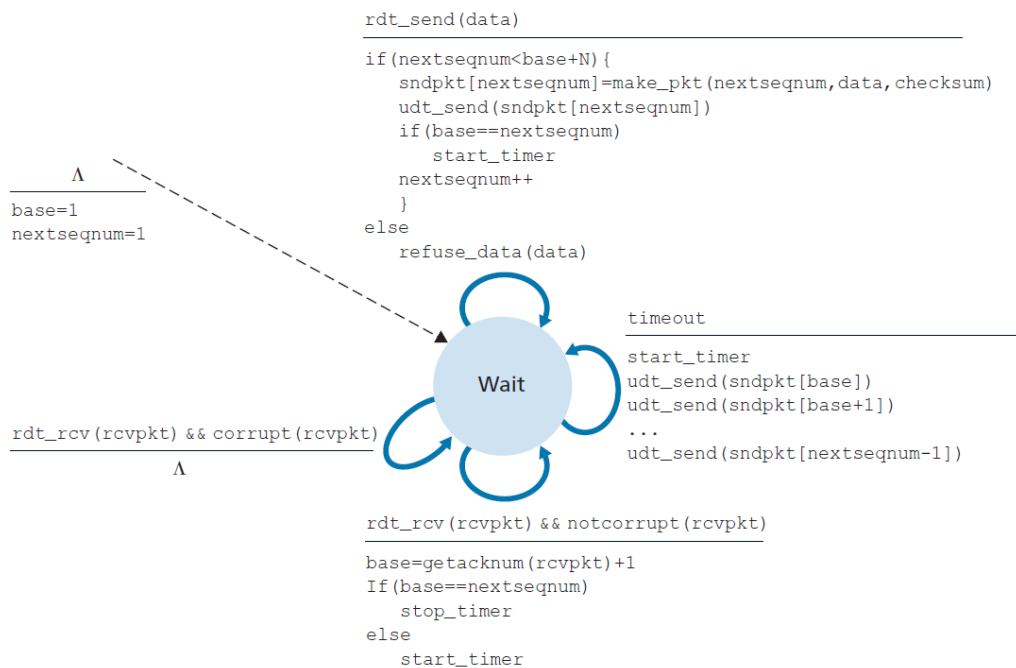


Figure 2: Extended FSM Description of GBN Sender

Receiver Behavior

The receiver maintains only one variable: **expectedseqnum**.

1. If the received packet matches **expectedseqnum** and is uncorrupted:
 - Deliver data to the upper layer.
 - Send ACK for that packet.
 - Increment **expectedseqnum**.

2. Otherwise, discard it and resend the most recent ACK.

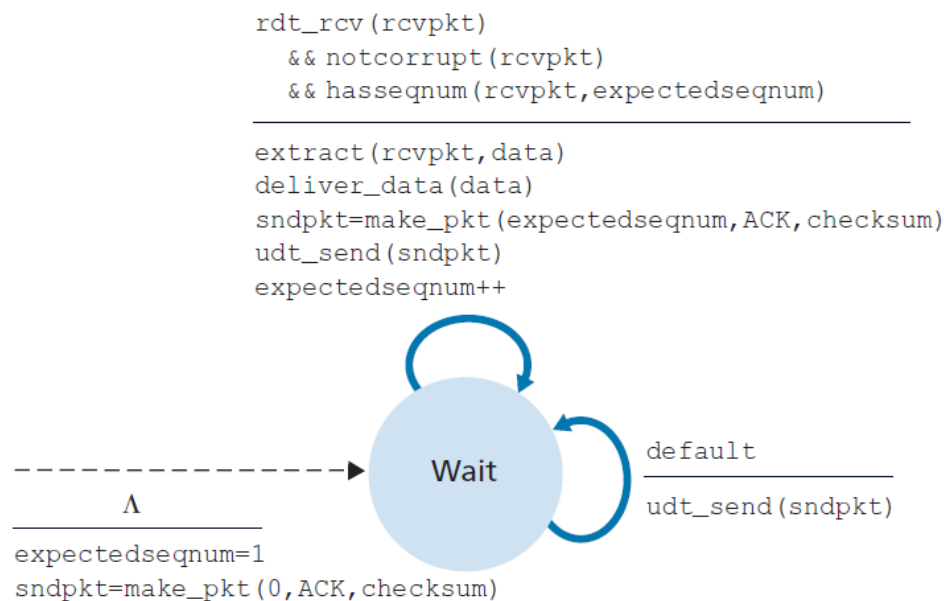


Figure 3: Extended FSM Description of GBN Receiver

Note: GBN uses **cumulative ACKs**. Out-of-order packets are discarded, simplifying the receiver but possibly wasting bandwidth.

Example: GBN Operation

Let $N = 4$. The sender transmits packets 0–3 and waits for ACKs. If packet 2 is lost, packets 3–5 are discarded at the receiver as “out-of-order.” After timeout, the sender retransmits all unacknowledged packets (2–5).

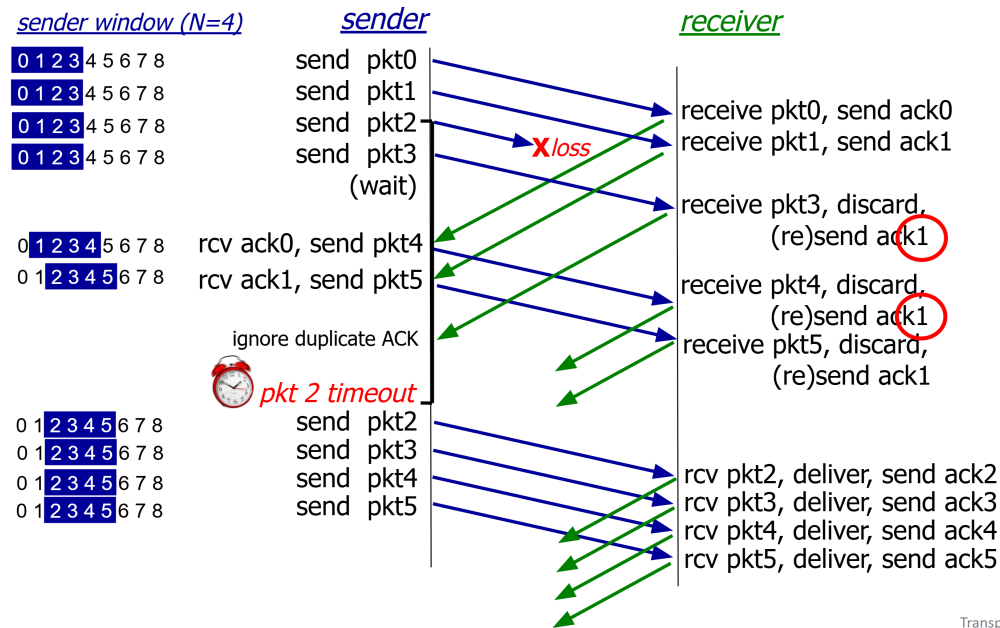


Figure 4: Operation of Go-Back-N with Window Size 4

Observation: While GBN fills the pipeline efficiently, a single loss triggers multiple re-transmissions, wasting bandwidth on unreliable links.

Summary

Highlights:

- Multiple unacknowledged packets allowed (up to window size N).
- Sliding window governs sender transmission.
- Uses cumulative ACKs.
- Receiver discards out-of-order packets.
- Timeout $\Rightarrow$ retransmit all pending packets.

Advantages:

- Better utilization than Stop-and-Wait.
- Simple receiver logic.

Disadvantages:

- Multiple retransmissions if errors occur.

In summary: GBN achieves high throughput through pipelining but suffers from redundant retransmissions—leading to improved designs like the **Selective Repeat (SR)** protocol.